

The Mathematical Intuition of Deep Learning

Wade K. Copeland

May 02, 2026

Why Learn the Mathematics of Deep Learning?

Learning the math behind a statistical method is like learning the secret to a magic trick. Once you know the math, you can never be fooled by the same trick again.

The Deep in Deep Learning

- ▶ Deep learning always refers to deep neural networks.
- ▶ A deep neural network is a neural network with more than two layers.
- ▶ ChatGPT Version 4 is reported to have 1.8 trillion parameters across 120 layers¹!

Preliminaries

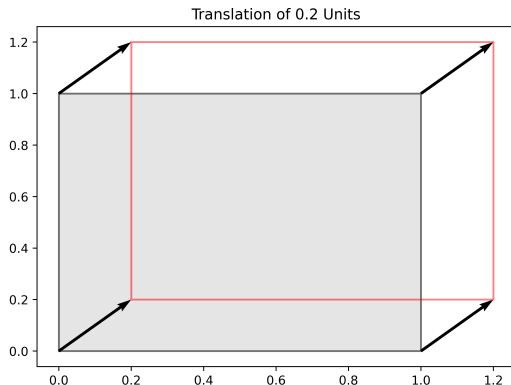
- ▶ This presentation is aimed at the someone who wants to learn about deep learning without having a background in sophisticated mathematics.
- ▶ We briefly introduce some concepts from *linear algebra* and *differential calculus*, but will focus on an intuitive understanding.

Matrices

- ▶ A *matrix* is a grid of numbers with N rows and P columns denoted $X_{N \times P}$.
 - ▶ $X_{2 \times 2} = \begin{bmatrix} x_1 & x_2 \\ x_3 & x_4 \end{bmatrix}$ is a 2×2 matrix.
 - ▶ $X_{1 \times 1} = [x_1]$ is a 1×1 matrix, also called a *scalar*.
 - ▶ $X_{1 \times 2} = [x_1 \ x_2]$ is a 1×2 matrix, also called a *vector*.

Matrix Translation

- ▶ *Matrix translation* is the addition/subtraction of a constant to every element of a matrix.
- ▶ The columns of $X_{2 \times 6} = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$ describe a square in 2-dimensional space. We can translate the square by 0.2 units by adding 0.2 to each element of $X_{2 \times 6}$.



Matrix Transformation

- ▶ *Matrix transformation* is achieved through the matrix-specific operation called the *dot product* where the rows of X and the columns of Y are multiplied and summed.

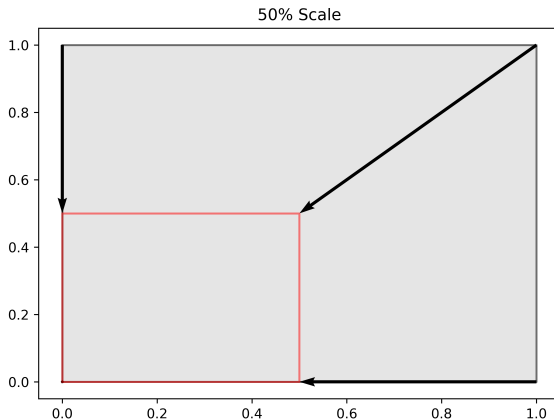
- ▶ The dot product of

$$X_{1 \times 2} \cdot Y_{2 \times 1} = [x_1 \quad x_2] \cdot \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = [x_1 y_1 + x_2 y_2]$$

- ▶ The dot product allows us to *scale* and *rotate* the feature matrix.

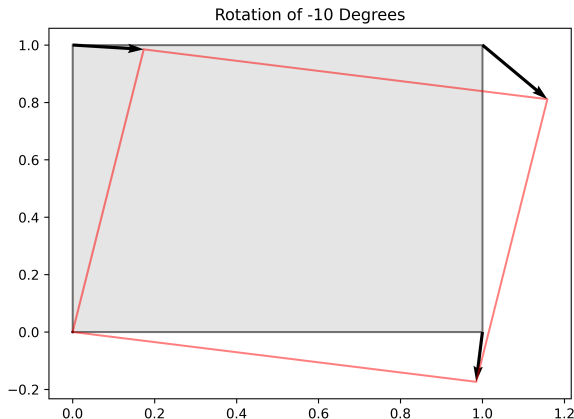
Matrix Transformation: Scale

- $X_{2 \times 6}$ can be scaled by 50% taking the dot product of $\begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$ and $X_{2 \times 6}$.



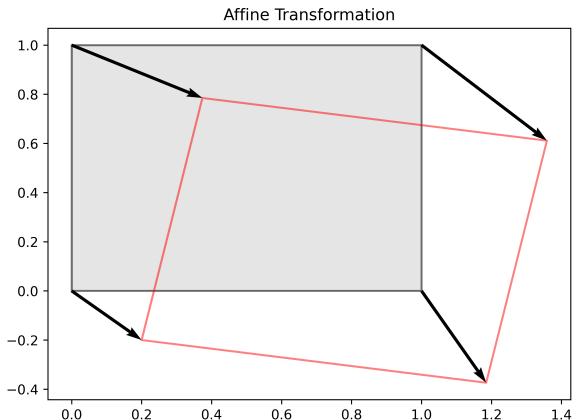
Matrix Transformation: Rotation

- $X_{2 \times 6}$ can be rotated by negative 10 degrees by taking the dot product of $\begin{bmatrix} \cos(0.175) & -\sin(0.175) \\ \sin(0.175) & \cos(0.175) \end{bmatrix}$ and $X_{2 \times 6}$.



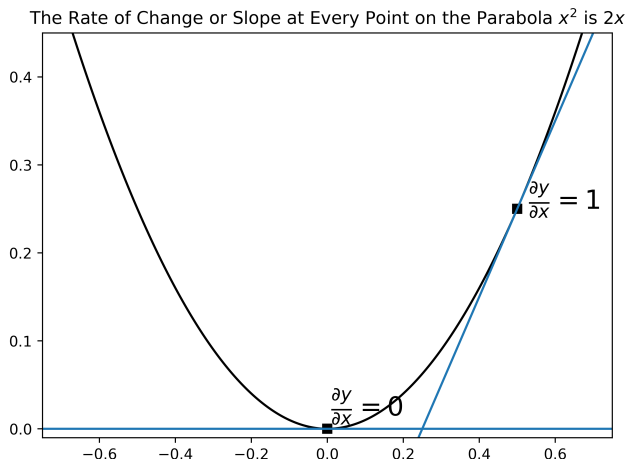
Matrix Transformation: Affine

- An *affine transformation* of $X_{2 \times 6}$ combines *transformation* and *translation*. For example, combining the rotation transformation of -10 degrees and translation of 0.2 units.



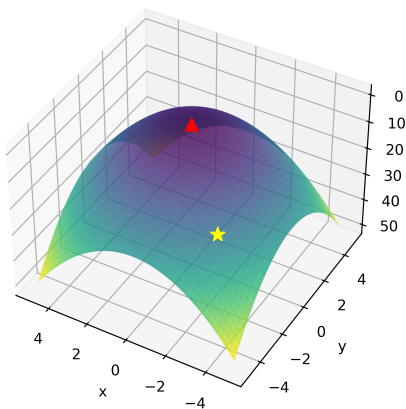
Rate of Change: The Derivative

- A *derivative* measures the rate of change or slope at any point denoted by $\frac{\partial y}{\partial x}$, which is read “the change in y with respect to the change in x .”



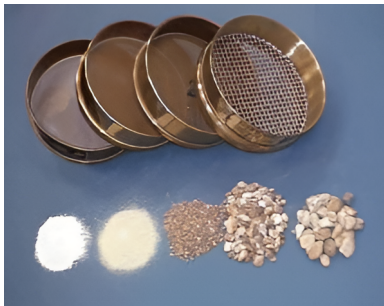
Rate of Change: The Gradient

- The *gradient* is a generalization of the derivative that measures the rate of change or slope in every direction. For example, if we have the surface $z = x^2 + y^2$ the slope in the x direction is $\frac{\partial z}{\partial x} = 2x$ and the slope in the y direction is $\frac{\partial z}{\partial y} = 2y$.



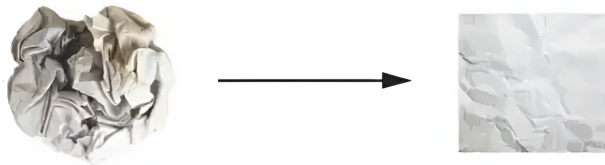
Deep Learning Intuition I

- ▶ A neural network is an information sieve that takes as input the outcomes, and the features we want to use to predict the outcomes².
- ▶ At the top of the sieve is the most granular representation of the data, and for each successive layer down, the data becomes more and more refined, that is, better at predicting the outcome².



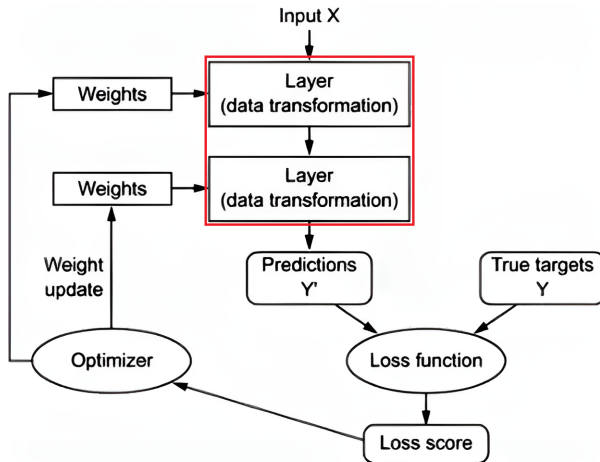
Deep Learning Intuition II

- Consider a blue and red piece of paper crumpled together and your goal is to uncrumple them. Each movement of your fingers to uncrumple the papers is like a layer of a neural network, until finally at the last layer the colors are fully separated².



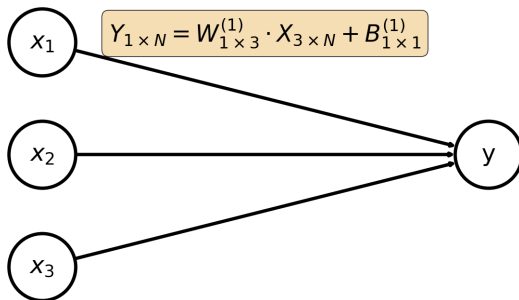
Deep Learning Roadmap: Layers

- ▶ The full process of fitting a neural network is shown below. As a first step, we will explain the mathematical representation of the layers².



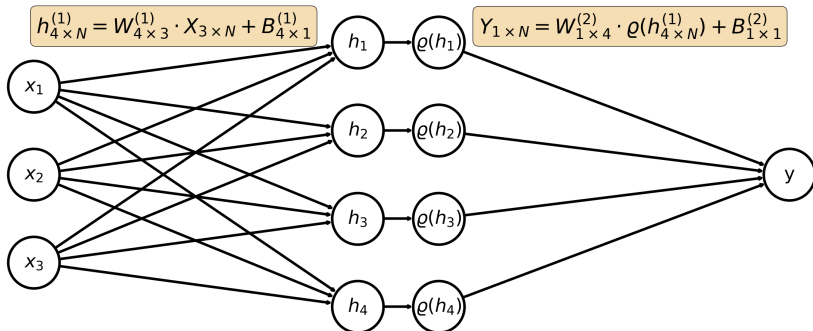
Representation of a Single-Layer Neural Network

- ▶ A single layer neural network is represented by the affine transformation $Y = W \cdot X + B^3$.



Representation of a Two-Layer Neural Network

- ▶ A two layer neural network includes a layer of *neurons* between the features and the outcome. These are called *hidden neurons* because they are not observed in the input data³.

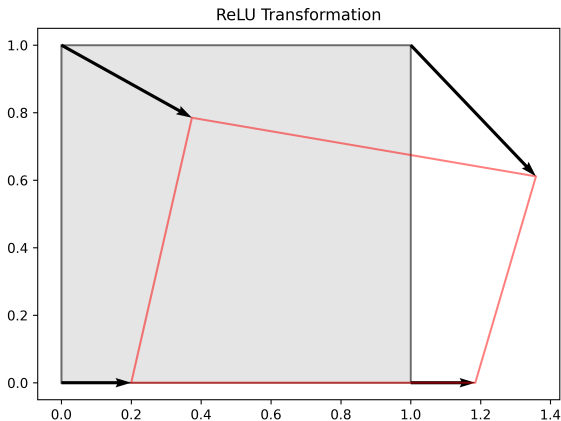


What Is $\varrho(h)$ and Why Do We Need It?

- ▶ The original affine transformation that defined each layer is linear, meaning that the output falls along a line.
- ▶ The function $\varrho(h)$, also called an *activation function*, is a special kind of transformation called a non-linear transformation that allows the layer to take on many forms that are not linear³.
- ▶ Without an activation function between layers, a deep neural network is actually a single-layer neural network in disguise²!

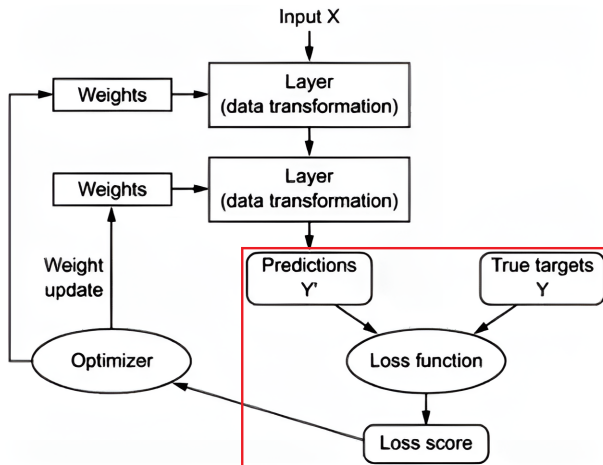
Rectified Linear Unit (ReLU)

- ▶ A common activation function is the *ReLU* transformation which replaces all negative values with 0.



Deep Learning Roadmap: Loss

- ▶ After a neural network is fit, we need a way to measure how good it is at predicting the outcome of interest. This is called a *loss function*².



Measures of Loss

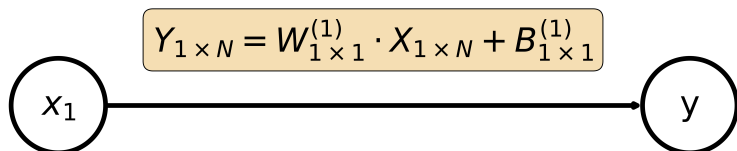
- ▶ For a numeric outcome a common measure of loss is the *mean squared error*, which measures the average of the squared deviations from the observed outcomes and our predictions of them⁴.

$$e = \frac{1}{n} \sum_{i=1}^n (y - y')^2$$

- ▶ There are many other measure of loss, such as *categorical cross entropy*, which can be used to measure loss when the outcome is categorical².

Forward Pass I

- ▶ Consider the most basic of neural networks, a single feature used to predict a single outcome.
- ▶ The *forward pass* calculates the predictions of the outcome from the input data using the values of W and B^2 .

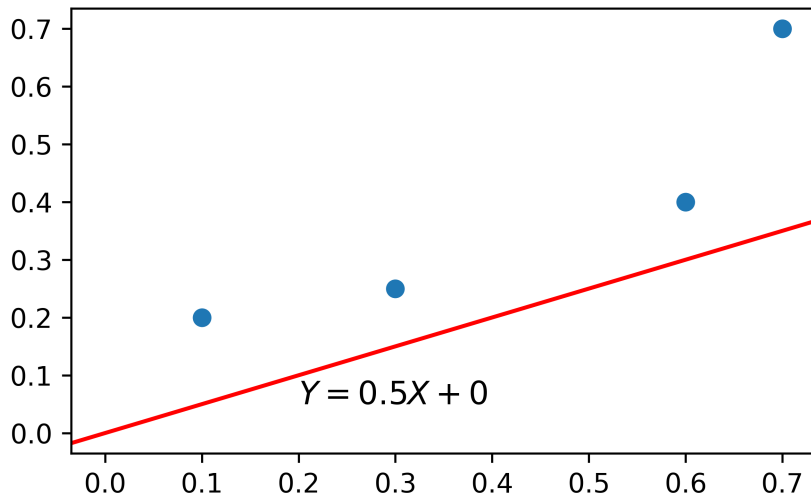


Forward Pass II

- ▶ Let $Y = [0.2 \ 0.25 \ 0.4 \ 0.7]$, $X = [0.1 \ 0.3 \ 0.6 \ 0.7]$, $W = [0.5]$, $B = [0]$.
- ▶ The initial predictions from the first forward pass are
$$Y' = W \cdot X + B = [0.5] \cdot [0.2 \ 0.25 \ 0.4 \ 0.7] + [0] = [0.05 \ 0.15 \ 0.3 \ 0.35].$$
- ▶ The calculated loss using the MSE is $e = \frac{1}{4}((0.2 - 0.05)^2 + (0.25 - 0.15)^2 + (0.4 - 0.3)^2 + (0.7 - 0.35)^2) = 0.0412$.

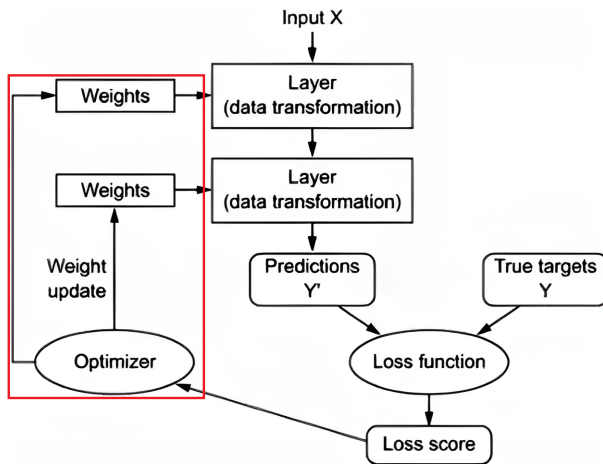
Forward Pass III

MSE After the First Forward Pass: 0.0412



Deep Learning Roadmap: Optimization

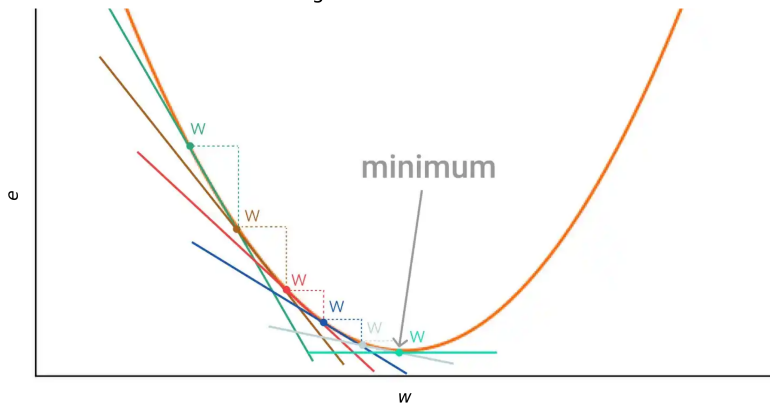
- The optimization step has the goal of *learning* the values of W and B that minimize the MSE^2 .



Gradient Descent

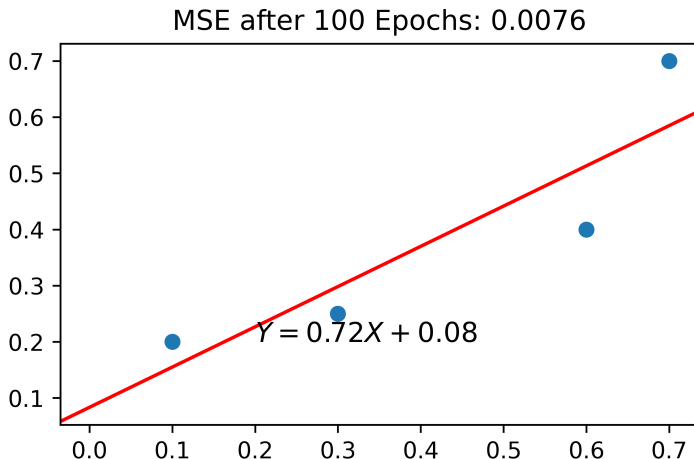
- ▶ Optimization is done numerically through an algorithm called *gradient descent*.
- ▶ Gradient descent works by stepping down the graph of our data in the opposite direction of the gradient⁵.

The Gradient Descent Algorithm to Find the Minimum of a Parabola



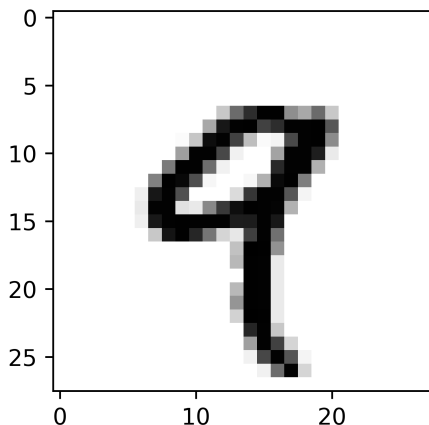
Backward Pass

- ▶ Adjusting the values of W and B using gradient descent is called a *backward pass*².
- ▶ Each cycle of a forward pass followed by a backward pass is called an *epoch*².



A Real Example: Digit Image Classification

- ▶ One of the best use cases for neural networks is image classification.
- ▶ We will use the MNIST data set that is 70,000 images of hand-written digits 0-9 that are 28x28 pixels⁶.



A Real Example: Data Setup

- ▶ For analysis each image map of 28×28 pixels needs to be converted into a vector of size $28 * 28 = 784$.
- ▶ The images are separated into a set of 60,000 images for training the model and a set of 10,000 images for testing the prediction accuracy in unseen data.

```
from tensorflow.keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = (
    mnist.load_data(path='mnist.npz')
)

train_images = train_images.reshape((60000, 28 * 28))
test_images = test_images.reshape((10000, 28 * 28))
```

A Real Example: Model Setup

- ▶ The first layer of the neural network contains 512 hidden neurons and uses the ReLU activation function.
- ▶ The output layer of the neural network is a 10-way Softmax classification layer that returns the probability that each digit belongs to a specific class.

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")
])
```

A Real Example: Model Compilation

- ▶ The **loss** argument specifies how the model loss is calculated at the end of each forward pass.
- ▶ The **optimizer** argument specifies the optimization algorithm used to update the parameter values for the backward pass.
- ▶ The **metric** argument specifies how we will assess the model performance.

```
model.compile(  
    loss="sparse_categorical_crossentropy",  
    optimizer="rmsprop",  
    metrics=["accuracy"])
```

A Real Example: Model Fit

- ▶ In the test data, the fitted neural network correctly classified each digit greater than 95% of the time!
- ▶ The difference in the accuracy of the training data and test data is an example of *overfitting*².

```
_ = model.fit(  
    train_images, train_labels, epochs=5, batch_size=128, verbose = 0)  
  
train_acc = model.evaluate(train_images, train_labels, verbose = 0)[1]  
test_acc = model.evaluate(test_images, test_labels, verbose = 0)[1]  
  
train_acc
```

```
0.9808499813079834
```

```
test_acc
```

```
0.9646999835968018
```


Summary

- ▶ This presentation peeled back the layers of what appears to be the *magic* of deep learning.
 - ▶ We discussed how *layers* in a neural network are like sieves for creating increasingly purified information.
 - ▶ We showed that the mathematical representation of each layer is just a combination of *matrix translation* and *matrix transformation*.
 - ▶ We explained that the purpose of the *activation function* applied to each layer is to detect non-linearity in the output function.
 - ▶ We showed how after each *forward pass* from the input to the output of the model, how the parameters in each layer are updated in the *backward pass* using *gradient descent* and *backpropagation*.
 - ▶ We concluded by presenting an example of fitting a neural network model using real data.

References I

1. Arya, N. (2023, July 19). *GPT-4 details have been leaked!*
<https://www.kdnuggets.com/2023/07/gpt4-details-leaked.html>
2. Chollet, F. (2021). *Deep learning with python*. Simon; Schuster.
3. Berner, J., Grohs, P., Kutyniok, G., & Petersen, P. (2021). The modern mathematics of deep learning. *arXiv Preprint arXiv:2105.04026*, 78.
4. Fox, J. (2008). *Applied regression analysis and generalized linear models*. Sage publications.
5. Bento, C. (2021, June 2). *Stochastic gradient descent explained in real life*.
<https://towardsdatascience.com/stochastic-gradient-descent-explained-in-real-life-predicting-your-pizzas-cooking-time-b7639d5e6a32>
6. Chollet, F. et al. (2015). *Keras*. <https://keras.io>.